NES AI FINANCE & INSURANCE INTELLIGENCE PLATFORM

Software Architecture Design

Document ID: NES-ARCH-004 | **Version:** 1.0

Build approach: Cline-assisted development • Local-first Windows application

Scope: Personal research, paper trading and insurance document assistance

System layers, domains, data flow and local-first boundaries

1. Architecture goals

- Keep UI, domain calculations, provider integrations and storage separate.
- Keep the initial prototype independent from real APIs and sensitive user data.
- Make external market-data providers replaceable through adapters.
- Make AI an orchestration/explanation layer, not the source of truth for calculations.
- Preserve the local-first design and testable module boundaries.

2. Logical architecture

Layer	Responsibilities
Frontend	Navigation, forms, charts, user interactions, accessibility and visual states.
API layer (later)	Validation, request/response contracts, error mapping and access boundaries.
Domain services	Trading calculations, backtests, paper simulation, policy comparisons and document workflows.
Adapters	Authorised market-data provider, PDF/OCR engines, local model interface.
Storage	Database metadata, local document files, reports, backups and audit events.

Layer	Responsibilities
AI orchestration	Retrieval, summarisation, explanation, source linking and uncertainty handling.

3. Recommended technology

- Frontend: Next.js + TypeScript + Tailwind + shadcn/ui; use the existing stack if project already created.
- Motion: GSAP + ScrollTrigger for complex timelines; CSS for basic interactions.
- 3D: Three.js + React Three Fiber, lazy-loaded with a static fallback.
- Backend later: Python 3.11 + FastAPI + Pydantic + SQLAlchemy + Alembic.
- Data processing later: Pandas, NumPy, scikit-learn and XGBoost where justified.
- Database: PostgreSQL as intended primary; SQLite for a small initial prototype.
- Local AI later: Ollama and a compatible quantized model, after evaluating memory and performance.

4. Suggested repository

nes-ai-finance/
■■■ frontend/ (app, components, hooks, lib, styles, types)
■ ■■■ components/ (shell, ui, motion, three, trading, insurance, copilot)
■■■ backend/app/ (api, core, db, data_sources, trading, backtesting, paper_trading, insurance, documents, ai, reports)
■■■ data/ (market, policies, reports, backups)
■■■ tests/
■■■ scripts/
■■■ docs/

5. Domain boundaries

- Trading domain owns indicator, backtest, portfolio and simulation calculations.
- Insurance domain owns policy records, extraction review and comparison logic.
- Document service handles file parsing and extracted field provenance.
- AI service calls validated domain services and retrieves authorised records; it does not invent calculations.
- UI components render state; they do not duplicate financial business logic.

6. Data flow

Market provider → provider adapter → validation/freshness metadata → domain service → stored result where appropriate → API contract → UI. Policy PDF → local extraction/OCR → extracted fields with confidence/provenance → user review → verified record → comparison or reminder. Copilot → user query → context retrieval → domain calculation/retrieval → answer with sources and limitations.

7. Security and resilience

- Provider credentials remain on backend/local configuration, never in client bundles.
- Sensitive document text must be excluded from normal diagnostic logs.
- Provide explicit backup, restore and deletion flows before storing real personal documents.
- Handle provider timeout, stale feed, partial data, OCR failure and local model unavailability.
- Three.js failure must not block the core application.

8. Architecture decisions to record

- Why Next.js or another frontend stack was selected.
- Why PostgreSQL vs SQLite for each stage.

- Chosen market data provider and its licence/latency constraints.
- Chosen OCR and local LLM options and their measured limitations.
- Storage path, backup format, encryption approach and data retention.